Projet d'Introduction à la Théorie des Langages de Programmation

ITLP

—Licence Informatique, 2ème année, 2ème semestre—

Ce projet est à réaliser en binôme. Il est à rendre au plus tard le 24 avril 2026 avant 20h par le dépôt d'une archive au format `zip` à l'emplacement prévu sur UPdago. **Aucun délai ne peut être accordé**.

Vous pouvez constituer votre groupe comme vous le souhaitez, mais **l'inscription dans un groupe sur UPdago est obligatoire** (voir dans l'onglet « Le projet » sur UPdago).

Les fichiers relatifs au projet sont organisés selon une arborescence correspondant aux exercices. Il vous faut la respecter. L'évaluation tiendra compte de la qualité de vos spécifications OCamllex et OCamlyacc et de la qualité de votre code ainsi que de son organisation, sa présentation et son commentaire.

Votre travail doit être personnel, produit par votre intelligence naturelle. Vous devez être capable d'en expliquer chaque élément.

Développement d'un mini interpréteur pour le langage Logo

Le langage Logo est un langage de programmation développé, à la fin des années 1960, pour l'apprentissage de la programmation particulièrement envers les écoliers. Vous pouvez trouver des ressources à cet endroit `https://el.media.mit.edu/logo-foundation/what_is_logo/history.html` et sur la page Wikipedia `https://fr.wikipedia.org/wiki/Logo_(langage)`.

Une des principales caractéristiques de Logo est sa partie graphique qui permet de manipuler un curseur, une « tortue » qui peut laisser une trace ou non dans une fenêtre graphique. Les programmes consistent ainsi à principalement dessiner des figures comme sur la figure 1.

L'objectif du projet est de développer un interpréteur pour un noyau de ce langage afin de mettre en œuvre les notions vues en cours/TD/TP. Afin de simplifier le développement, notre langage ne va pas suivre exactement la syntaxe du langage Logo, mais s'en inspirera fortement. On va travailler sur un sous-ensemble simplifié comprenant les différentes commandes de dessins, la définition de variables, les expressions arithmétiques en nombres entiers, les séquences d'instructions, la boucle et la conditionnelle.

Pour illustrer le langage que vous allez définir, voici le programme qui a dessiné les tracés de la figure 1.

```
-- Des etoiles et une maison
Efface ;
```

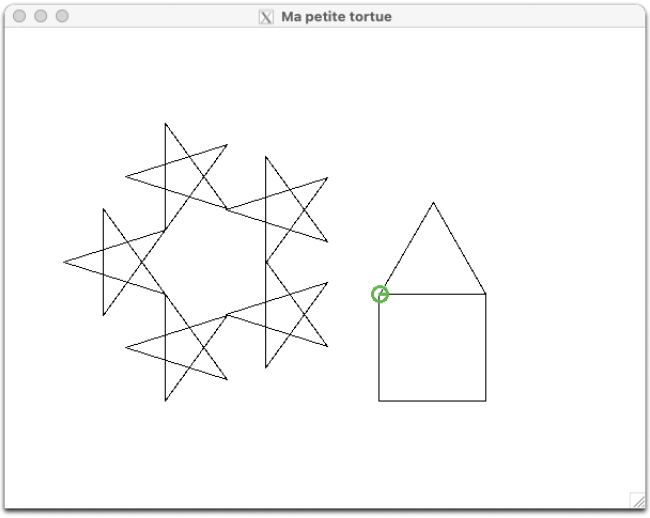

FIGURE 1 – Un exemple de dessin obtenu par un tracé de la tortue.

```
-- Positionnement de la tortue
Avance 150 ; Gauche 90 ; Avance 100; Dessine ;
-- Dessin des etoiles
Repete 5 [Deplace ; Avance 160; Dessine; Repete 6 [Avance 100; Droite 144]] ;
-- Positionnement de la tortue
Deplace; Droite 90 ; Avance 200 ; Gauche 90; Dessine ;
-- Dessin de la maison
Def taille 100 ;
Repete 4 [Avance :taille; Droite 90] ;
Deplace ; Avance :taille ; Droite 90 ; Dessine ;
Repete 3 [Avance :taille; Gauche 120] .
```

Éléments du langage

Le langage que l'on souhaite définir comprend :

- Des entiers (positifs ou négatifs) : `42, +23, -67`,
- Des variables dont les noms sont des chaînes de caractères en minuscules ou majuscules et sans accent, pouvant comprendre des chiffes et le caractère `'_'`, commençant par une minuscule ou le caractère `'_'` : `toto, _tutu14, truc_13, machin_et_bidule`.
  Une variable est définie à l'aide du mot-clef `Def` suivi du nom de la variable suivi de sa valeur : `Def toto 42`, `Def truc_a (23*6)`. En mettant le caractère `:` devant le nom d'une variable, on accède à sa valeur : `Def toto (2 * :truc_13)`, `Avance (23 + :toto)`,
- Les expressions arithmétiques sont complètement parenthésées et comprennent les opérations d'addition, de soustraction, de multiplication, de division et de reste de la division (le « modulo »).
- Des commandes de contrôle de la tortue : `Avance`, `Recule`, `Gauche`, `Droite` suivies d'une expression permettent respectivement de faire avancer ou reculer la tortue, de l'orienter

vers un cap à gauche et à droite (voir la figure 2). Les commandes `Efface`, `Dessine`, `Deplace` permettent d'effacer la fenêtre graphique et de réinitialiser la tortue, de laisser une trace quand la tortue se déplace, de ne pas laisser de trace quand la tortue se déplace. Dans la fenêtre graphique, la position de départ de la tortue est le point de coordonnées $(0, 0)$ (en bas à gauche) et son cap est 0 (i. e. elle avance vers la droite). Le cap de la tortue indique la direction de son prochain déplacement vers l'avant. La tortue est matérialisée par un cercle dont on a dessiné un rayon. Ce rayon indique le cap de la tortue.

- La boucle, dont le mot-clef est `Repete`, prend en arguments une expression qui indique combien de fois le corps de la boucle va être exécuté et une suite d'instructions encadrée par des crochets `'['` et `']'`. Dans une suite d'instructions, les instructions sont séparées par un point-virgule `';'`, par exemple : `Repete 4 [Avance :taille; Droite 90]`.
- La conditionnelle s'écrit `Si cond Alors instr1 Sinon instr2 FinSi`. Où `cond` est une condition comparant deux expressions avec les opérateurs `=`, `>` et `<` et `instr1` et `instr2` sont des suites d'instructions. Il y a toujours la partie « `Sinon` ».
- Pour une suite d'instructions, les instructions sont séparées par le caractère `';'`.
- Les phrases du langage se terminent par le caractère `'.'`. C'est à ce moment que l'interprétation est déclenchée. Ainsi une phrase (un programme) peut être écrit sur plusieurs lignes).
- Les commentaires commencent par `--` et se terminent par un retour à la ligne.

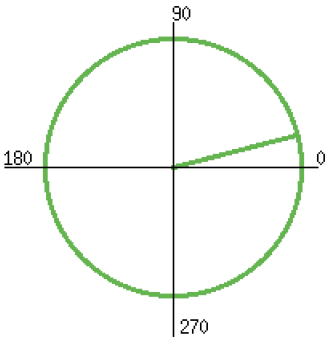

FIGURE 2 – Les valeurs des caps de la tortue. Le rayon dessiné indique le cap de la tortue.

Fichiers fournis

Les fichier fournis sont :

- Le fichier `env.ml` qui définit un module `Env` pour gérer les environnements où sont stockées les valeurs des variables (comme en cours et en TP).
- Le fichier `geometry.ml` qui définit un module `Geometry` où des fonctions de calculs géométriques utilisés pour les déplacements de la tortue sont définies.
- Le fichier `turtle.ml` qui définit un module `Turtle` où sont définis le type `turtle` et les opérations de manipulation d'une valeur du type `turtle`. Ce module permet d'utiliser une version non graphique de notre langage.

- Le fichier `turtle_graphic.ml` qui définit un module `Turtle_graphic` pour la manipulation d'une fenêtre graphique où sont représentés la tortue et les traces qu'elle dessine. Ce module fonctionne avec le module `Graphics` de OCaml. Il est installé par défaut sur les machines des salles de TP et vous pouvez l'installer par la commande `opam install graphics` sur votre machine personnelle. Des informations sur le module `Graphics` se trouvent à cet endroit `https://github.com/ocaml/graphics`. Il n'est pas obligatoire d'avoir une version graphique pour la réalisation du projet (mais le résultat est plus amusant...).
- Le fichier `logo_ast.ml` qui définit le module `Logo_ast` où sont définis les types qui permettent de représenter la syntaxe abstraite du langage. Dans ce fichier on peut utiliser ou non la version graphique de la tortue.
- Le fichier `repl_logo.ml` où est définie la boucle d'interaction à utiliser pour votre projet. Dans ce fichier on peut utiliser ou non la version graphique de la tortue.
- Le fichier `Makefile` qui permet de compiler vos fichiers et de créer un fichier exécutable avec l'outil `make`. Le fichier `Makefile` permet de compiler avec ou sans la version graphique de la tortue. Pendant la phase de mise au point, il est conseillé de ne pas utiliser la version graphique de la tortue.

Les exercices

▸ **Exercice 1. (La base)** *Dans un premier temps, on va mettre en place la partie du langage qui ne comprend que les entiers et les variables, la définition des valeurs d'une variable, les commandes qui manipulent la tortue et les suites d'instructions. Il n'y a pas la possibilité de gérer les commentaires.*

1. *Dans le fichier `logo_ast.ml` du répertoire `Exo01`, définissez les interpréteurs pour les valeurs des types abstraits des expressions `t_expr`, des commandes de la tortue `t_cmd` et des instructions `t_instr`. Testez vos fonctions d'interprétation dans l'interpéteur OCaml comme on a pu le faire en TP. Gardez le fichier obtenu sous le nom `logo_ast_exo01-1.ml`.*
2. *Dans un fichier `logo_lexr.mll`, définissez une spécification OCamllex reconnaissant les entiers (positifs ou négatifs), les noms des variables, les éléments lexicaux nécessaire à la reconnaissance de la définition de la valeur d'une variable et aux commandes `Avance`, `Recule`, `Gauche`, `Droite`, `Efface`, `Dessine`, `Deplace` ainsi que le caractère `'.'` qui termine une phrase. Dans un premier temps écrivez une version qui affiche le lexème reconnu et, si nécessaire, la valeur associée. Gardez ce fichier sous le nom `logo_lexr_exo01-2.mll`. Ensuite, modifiez le fichier `logo_lexr.mll` pour que les règles envoient les lexèmes correspondant aux mots reconnus.*
3. *Dans un fichier `logo_parser.mly` écrivez une spécification OCamlyacc qui définit la syntaxe du fragment du langage considéré dans cet exercice. Dans un premier temps affichez une chaîne de caractères qui indique la règle utilisée pour la reconnaissance d'une partie de phrase du langage. Gardez ce fichier sous le nom `logo_parser_exo01-3.mly`. Ensuite, modifiez le fichier `logo_parser.mly` pour que les actions associées à chaque règle envoie le morceau correspondant de l'arbre de syntaxe abstraite.*

4. *Generez l'analyseur lexical et l'analyseur syntaxique et assemblez ces fichiers avec le reste des fichiers pour créer une première version de l'interpréteur Logo.*
5. *Testez votre interpréteur sur les programmes suivants :*

```
Efface ;
Avance 200 ; Gauche 90 ; Avance 100; Dessine ;
Avance 100 ; Gauche 90; Avance 100; Gauche 90;
Avance 100; Gauche 90; Avance 100; Gauche 90 .

Efface ;
Avance 150 ; Gauche 90 ; Avance 200; Dessine ;
Def taille 100 ; Def demitaille 50 ;
Avance :taille ; Recule :demitaille ; Droite 90 ;
Avance :demitaille ; Droite 90 ;
Avance :demitaille ; Recule :taille .
```

Pour éviter d'écrire à nouveau tout le code lors de chaque essai d'interprétation, pensez à l'enregistrer dans un fichier. Pour la version non graphique, on peut envoyer tout un fichier à la boucle d'interaction avec la commande `./repl_logo < mon_fichier.logo`.

▸ **Exercice 2. (Les extensions)** *On souhaite maintenant ajouter les différents éléments du langage.*

1. *On commence par les commentaires. Modifiez l'analyseur lexical (i.e. la spécification dans* `logo_lexer.mll` *afin de pouvoir écrire des programmes ayant des commentaires. Ces commentaires sont bien sûr ignorés dès la phase d'analyse lexicale. Gardez le fichier sous le nom* `logo_lexer_exo02-1.mll`*Testez en utilisant le programme :*

```
-- carre sans boucle
Efface ;
-- on place la tortue au bon endroit
Avance 200 ; Gauche 90 ; Avance 100; Dessine ;
-- on avance et on tourne a angle droit quatre fois de suite
Avance 100 ; Gauche 90; Avance 100; Gauche 90;
Avance 100; Gauche 90; Avance 100; Gauche 90 .
```

2. *Modifiez les fichiers nécessaires pour ajouter les boucles à votre langage. Pour l'intepretation d'une boucle, vous allez avoir besoin de la fonction :*

```
let rec iterate n f a =
  if n > 0
  then iterate (n - 1) f (f a)
  else a
;;
```

qui permet d'itérer une fonction `f`, `n` *fois à partir d'une valeur initiale* `a`. *On a vu un mécanisme semblable en cours et en TP et vous pouvez aussi utiliser une boucle (attention à bien gérer les versions successives de l'environnement). Gardez les fichiers modifiés sous le format* `mon_fichier_exo02-2`. *Testez votre nouvelle version de l'interpréteur sur les programmes :*

```
Efface ;
Avance 150 ; Gauche 90 ; Avance 200; Dessine ;
Def taille 100 ;
Repete 4 [Avance :taille; Droite 90] ;
Deplace ; Avance :taille ; Droite 90 ; Dessine ;
Repete 3 [Avance :taille; Gauche 120] .

Efface ;
Avance 150 ; Gauche 90 ; Avance 100; Dessine ;
Repete 5 [Deplace ; Avance 160; Dessine; Repete 6 [Avance 100; Droite 144]] .
```

3. *Une étape supplémentaire est d'ajouter les opérations arithmétiques pour les expressions (totalement parenthésées) en prenant exemple sur ce que l'on a fait en TP. Les opérations sont l'addition, la soustraction, la multiplication, la division et le reste de la division (le « modulo »). Effectuez les modifications nécessaires et gardez les fichiers modifiés sous le format* `mon_fichier_exo02-3`*. Testez votre interpréteur sur les programmes :*

```
Efface ;
Avance 200 ; Gauche 90 ; Avance 200; Dessine ;
Def taille 150 ; Def nbfois 24 ;
Repete :nbfois [Droite (360 / :nbfois) ;
                Repete 3 [Avance :taille; Gauche 120]] .

Efface ;
Avance 200 ; Gauche 90 ; Avance 220; Dessine ;
Def i 1 ;
Repete 100 [Avance (3 * :i); Gauche 91; Def i (:i + 1)] .
```

▸ **Exercice 3. (Ajout de la conditionnelle)** *Le dernier élément que nous allons ajouter à notre langage est la conditionnelle. Pour cela nous avons besoin de pouvoir comparer deux expressions. Cela se fait avec les relations égale* `=`*, plus grand que* `>` *et plus petit que* `<`*.*

1. *Dans le fichier* `logo_ast.ml`*, en prenant modèle sur les expressions arithmétiques, ajoutez les types* `t_opcmp` *pour représenter les opérateurs de comparaison et* `t_bexpr` *pour représenter la syntaxe abstraite d'une comparaison.*
2. *Ajouter maintenant une fonction* `interp_bexpr` *d'interprétation des comparaisons.*
3. *On peut maintenant ajouter la conditionnelle au type* `t_instr`*, elle a besoin d'une comparaison et de deux instructions. Modifiez en conséquence, le fonction* `interp_instr` *afin d'interpréter une conditionnelle. Gardez le fichier* `logo_ast.ml` *obtenu sous le nom* `logo_ast_exo03-3.ml`*.*
4. *Voici un exemple de conditionnelle* `Si (:a > :b) Alors Avancer 30 Sinon Reculer 50 FinSi`*. Modifiez les fichiers* `logo_lexer.mll` *et* `logo_parser.mly` *afin de pouvoir écrire et interpréter des programmes contenant des conditionnelles comme :*

```
Efface ;
Avance 200 ; Gauche 90 ; Avance 220; Dessine ;
Def cotes 10 ;
```

```
Si (:cotes > 2) Alors Repete :cotes [Avance 60; Droite (360 / :cotes)]
Sinon Avance 60 FinSi .
```

Gardez les fichiers `logo_lexer.mll` *et* `logo_parser.mly` *obtenus sous les noms* `logo_lexer_exo03-4.mll` *et* `logo_parser_exo03-4.mly`*.*

Générez les interpréteurs lexical et syntaxique et construisez le nouvel interpréteur du langage. Testez votre interpréteur sur le programme précédent.